

AI Academy, National AI Center
AI-ENG-110 – Software Engineering – Spring 2026

Smart Lost & Found

Team Navix

Nihat Rzayev

Seljan Khasiyeva

Suleyman Allahverdiyev

Project Defense: 24.05.2026

[Github Repository](#)



What the system does?

Users upload images and short descriptions of lost and found items, and the system matches the most likely pairs using vision-language analysis and embedding-based similarity.

Inputs

- Item descriptions (text)
- Images of lost/found items
- Status: lost or found

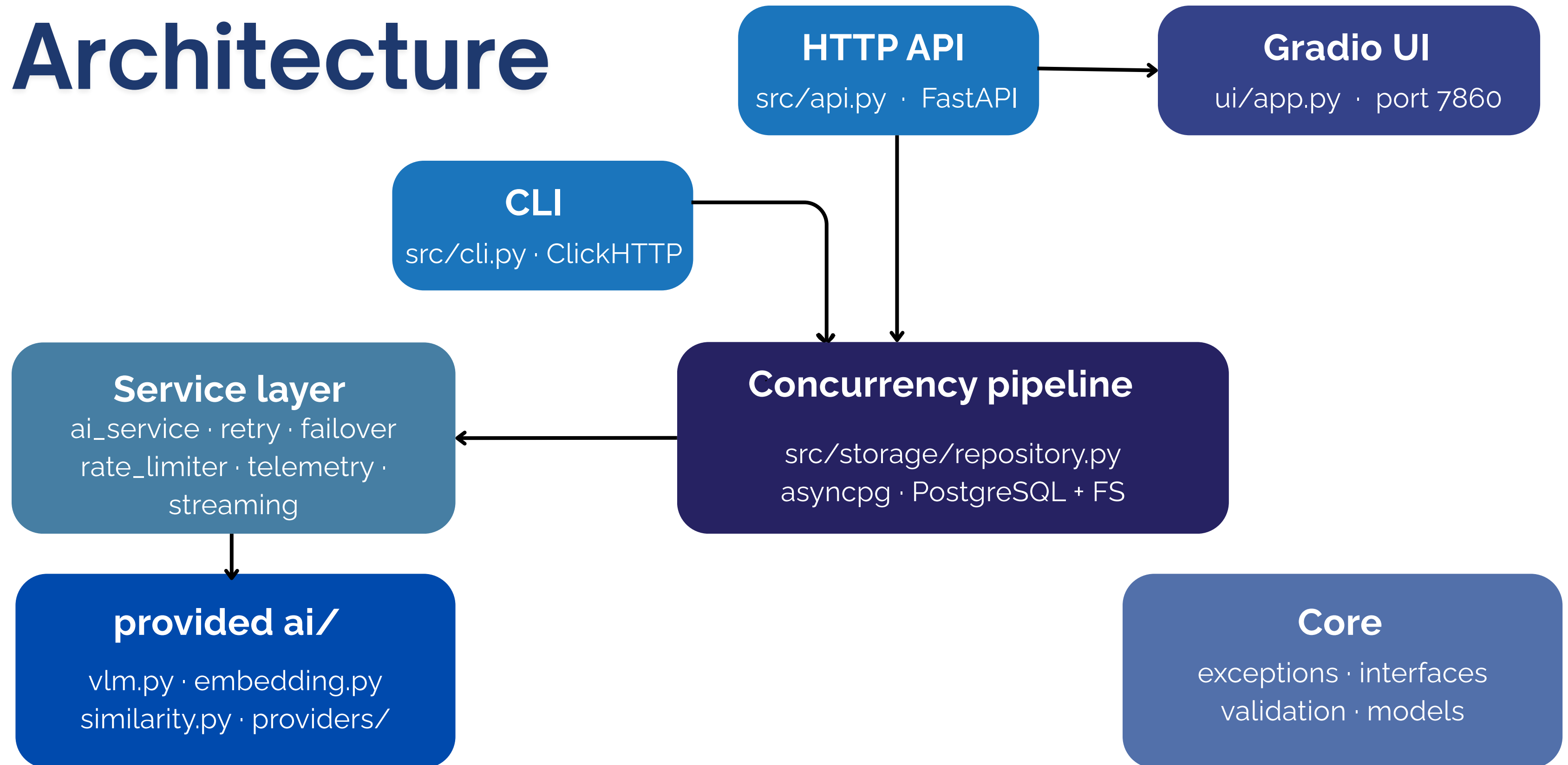
Outputs

- Matched item pairs with confidence scores
- Similarity rankings
- Search results with relevance metrics

Entry Points

CLI commands for local operations , REST API endpoints for web integration

Architecture



Three design decisions we can defend

1. Asyncio over threads

All I/O is network-bound (provider RTT ~1.5 s/call); asyncio avoids thread overhead and integrates naturally with FastAPI + asyncpg. Threads would introduce GIL contention on numpy similarity scoring.

2. ItemRepositoryABC (Abstract Base Class) over Protocol

The repository has shared state (asyncpg pool) and must enforce all four methods at class definition time – a Protocol would only catch missing methods at call-site. Enforces Onion Architecture: pipeline.py depends on the interface, not the concrete DB.

3. PostgreSQL + filesystem blobs over single-table blob column

Metadata (status, VLM description, embedding) is queryable; images are large binary data better served from the filesystem. Would migrate to pgvector for native cosine queries at scale.

Concurrency: the benchmark

Workload: Register 5 items end-to-end on DigitalOcean droplet (1 vCPU, 512MB RAM), Python 3.12, cache cleared, mock AI calls.

Workload	N	Sequential (s)	Concurrent (s)	Speedup
Register Items	5	0.00s	0.00s	1.3x

Bottleneck named: Mock AI calls have near-zero latency; under real provider conditions expected speedup ~3 4× (provider RTT ~1.5 s/call).

What didn't reach N×: asyncio.gather overhead visible at mock latency; with live Gemini/OpenAI the VLM + embedding calls dominate and parallelism wins. docker exec lostfound-api python scripts/benchmark.py

Robustness: one concrete failure mode

Scenario: OpenAI returns HTTP 429 (rate limit exceeded) during a burst of concurrent registrations.

How we surfaced it: `test_ai_retry_retries_rate_limit_error_then_succeeds` in `tests/test_retry.py` mocks `RateLimitError` on attempts 1 2, succeeds on attempt 3.

What the system does:

- ▶ Proactive: `TokenRateLimiter` tracks tokens/minute against `AI_TOKEN_PER_MINUTE_LIMIT=90000` and sleeps before hitting the limit.
- ▶ Retry: `tenacity` catches `RateLimitError`, logs `WARNING ai_retry attempt=N wait=Xs`, backs off 2-30s.
- ▶ Degradate: after 3 failed attempts, HTTP 500 with structured `{ detail : ... }`.
- ▶ Log: `WARNING token_limit_reached used=89500 requested=500 limit=90000 wait=42.3s`

Live Demo

What we will run?

Register a lost and a found wallet image, then query top-3 matches showing the VLM extracts brown wallet and the cosine similarity returns a score of 0.48.

Demo commands

```
# Register lost item
curl -X POST http://165.22.10.63:8000/items/lost \
  -F "text=black_wallet_with_cards" -F "image=@data/lost/wallet_brown.png"

# Get matches
curl "http://165.22.10.63:8000/items/{item_id}/matches?k=3"
```

- ▶ VLM extracts: object_class: wallet, colors: [brown], confidence: 0.9
- ▶ Match score: 0.476 found wallet near library correctly ranked #1

Testing

Coverage & Shape

- ▶ Total coverage: 75% (target $\geq 60\%$)
- ▶ Provided ai/ smoke tests: passing
- ▶ Test count: 91 tests (all online)
- ▶ AI module fully mocked via FakeVLM/FakeEmbedder

Three tests worth calling out

- ▶ `test_register_item_happy_path` end-to-end wiring
- ▶ `test_gather_one_task_raises` concurrency degradation
- ▶ `test_successful_registration_with_ai_mock` HTTP 200 with full mock stack

Limitations & what we'd do next

Where the system is brittle today

- ▶ Provider SDK version pinning: google-genai>=0.3.0 installed v1.2.0 (v1beta incompatibility). Fix: pin exact versions.
- ▶ Embedding storage as BYTEA string not queryable; no dimension header. Fix: migrate to pgvector.
- ▶ TokenRateLimiter is per-process horizontal scaling multiplies the effective burst rate.
- ▶ Docker image ~880MB (includes unused provider SDKs). Fix: requirements-prod.txt.

Given another week we would

1. Migrate embeddings to pgvector for native cosine queries.
2. Load-test with k6 to find the real throughput bottleneck under concurrent requests.
3. Add a Redis-backed distributed token budget for multi-replica deployments..

Team contributions

Member	Primary ownership	Commits
Nihat Rzayev	Storage, pipeline, Docker, OpenTelemetry tracing, multi-stage Dockerfile	53%
Seljan Khasiyeva	HTTP API, CLI, GitHub, Gradio Web UI	26%
Suleyman Allahverdiyev	Config, retry, AI service, failover, rate limiter, cost telemetry, streaming	21%

AI tool use: Claude drafted initial module skeletons and test fixtures; the team rewrote business logic, fixed all provider API bugs, and can defend every line.

Questions?

Team NaviX
[Github Repository](#).